1. Code
    a. 使用 getopt 來處理要設定的變數，有 thread 的數量&policy&priority、busy waiting 的時間 (全域變數)

```
42  int main(int argc, char* argv[]) {
43      int num_threads;// = 4;
44      //double time_wait;// = 0.5;
45      vector<string> policies;//{"FIFO", "NORMAL", "FIFO", "NORMAL"};
46      vector<int> prior;//{-1,10,-1,30};
47
48
49      /* 1. Parse program arguments */
50      int opt;
51      string token;
52      istringstream iss;
53
54      while ((opt = getopt(argc,argv, "n:t:s:p:")) != -1) {
55          switch(opt) {
56              case 'n':
57                  num_threads = stoi(optarg);
58                  break;
59              case 't':
60                  time_wait = stod(optarg);
61                  break;
62              case 's':
63                  iss.clear();
64                  iss.str(optarg);
65                  while (getline(iss, token, ',')) {
66                      policies.push_back(token);
67                  }
68                  break;
69              case 'p':
70                  iss.clear();
71                  iss.str(optarg);
72                  while (getline(iss, token, ',')) {
73                      prior.push_back(stoi(token));
74                  }
75                  break;
76              default:
77                 break;
78          } // end switch
79      } // end opt while
```

b. 設置 barrier(全域變數)，以便一次啟動全部的 thread

```
12      pthread_barrier_t barrier;
```

```
81          pthread_barrier_init(&barrier, NULL, num_threads+1);
```

c. 設定 cpu affinity 以及 thread 的 policy&priority

```
88      /* 3. Set CPU affinity */
89      int cpu_id = 2;
90      cpu_set_t cpuset;
91      CPU_ZERO(&cpuset);
92      CPU_SET(cpu_id, &cpuset);
93
94      for (int i = 0; i < num_threads; i++) {
95          /* 4. Set the attributes to each thread */
96          pthread_attr_t attr;
97          struct sched_param param;
98
99          pthread_attr_init(&attr);
100
101         int policy;
102         if (policies[i] == "FIFO") {
103             policy = SCHED_FIFO;
104         }
105         else {
106             policy = SCHED_OTHER;
107         }
108
109         pthread_attr_setinheritsched(&attr, PTHREAD_EXPLICIT_SCHED);
110         pthread_attr_setschedpolicy(&attr, policy);
111
112         param.sched_priority = prior[i];
113         pthread_attr_setschedparam(&attr, &param);
114
115         if (pthread_create(&threads[i], &attr, worker, &thread_id[i]) != 0) {
116             cerr <<  "Error!\n";
117         }
118
119         pthread_setaffinity_np(threads[i], sizeof(cpu_set_t), &cpuset);
120         pthread_attr_destroy(&attr);
121
122     }
```

d. 等所有 thread 都創建好了，一次執行全部，最後結束整個程式

```
124     /* 5. Start all threads at once */
125     pthread_barrier_wait(&barrier);
126
127     /* 6. Wait for all threads to finish  */
128     sleep(10);
129     pthread_barrier_destroy(&barrier);
130
131     return 0;
132 }
```

e. Thread function，印出 3 次 thread n is running，每次都等待 time_wait 時間(busy waiting)再繼續執行

```
30  void * worker(void *arg) {
31
32      pthread_barrier_wait(&barrier);
33      int *n = (int *)arg;
34      for(int i = 0; i < 3; ++i) {
35          cout << "Thread " << *n << " is running" << endl;
36          busyWait(time_wait);
37      }
38      return NULL;
39  }
```

f. Busy waiting function，會執行 t 秒後結束

```
void busyWait(double t) {
    auto startTime = chrono::high_resolution_clock::now();
    long long durationNanoSeconds = static_cast<long long>(t * 1e9);

    while(true) {
        auto currentTime = chrono::high_resolution_clock::now();
        auto elapsedTime = chrono::duration_cast<chrono::nanoseconds>(currentTime-startTime).count();
        if (elapsedTime >= durationNanoSeconds) {
            break;
        }

        int x = 0;
        for (int i = 0; i < 10000; ++i) x+=i*i;
    }
}
```

2. Result of ./sched_demo -n 3 -t 1.0 -s NORMAL,FIFO,FIFO -p -1,10,30

```
sunruru@sunruru-virtual-machine:~$ sudo ./sched_demo_311553059 -n 3 -t 1.0 -s NORMAL,FIFO,FIFO -p -1,10,30
[sudo] password for sunruru:
Thread 2 is running
Thread 2 is running
Thread 2 is running
Thread 1 is running
Thread 1 is running
Thread 1 is running
Thread 0 is running
Thread 0 is running
Thread 0 is running
```

因為 FIFO 是 real-time，所以 policy 是 FIFO 的會先執行，且沒有執行完不會被 preempt。thread1 跟 2 都是 FIFO，這時候是 priority 數值較大者先執行。所以造成如圖中的結果。

3. Result of ./sched_demo -n 4 -t 0.5 -s NORMAL,FIFO,NORMAL,FIFO -p -1,10,-1,30

```
sunruru@sunruru-virtual-machine:~$ sudo ./sched_demo -n 4 -t 0.5 -s NORMAL,FIFO,NORMAL,FIFO -p -1,10,-1,30
Thread 3 is running
Thread 3 is running
Thread 3 is running
Thread 1 is running
Thread 1 is running
Thread 1 is running
Thread 2 is running
Thread 0 is running
Thread 2 is running
Thread 0 is running
Thread 2 is running
Thread 0 is running
```

如上題所述，FIFO 會優先執行且不會被 preempt，其他兩個 NORMAL 的 thread 則會輪流使用 CPU，所以會看到 thread2 跟 0 輪流 print。

4. Busy-waiting function

```
void busyWait(double t) {
    auto startTime = chrono::high_resolution_clock::now();
    long long durationNanoSeconds = static_cast<long long>(t * 1e9);

    while(true) {
        auto currentTime = chrono::high_resolution_clock::now();
        auto elapsedTime = chrono::duration_cast<chrono::nanoseconds>(currentTime-startTime).count();
        if (elapsedTime >= durationNanoSeconds) {
            break;
        }

        int x = 0;
        for (int i = 0; i < 10000; ++i) x+=i*i;
    }
}
```

使用現在的時間減去開始的時間，當到達 busy-waiting 的秒數離開 while(true)，而底下的 for 則是為了確保這個程式有在使用 cpu，不會因 compiler optimization 而不見或跑的時間不夠長。